PROGRAMMING IN JAVA

ASSIGNMENT 5

CO5 – DATA PERSISTENCE ENGINEERING

Files and Databases

Madaraboina Gayathri

Register Number: 192472074

Java Version: 26.0.1

MySQL Version: 8.0.46

1. Student Details

Item	Details
Name	Madaraboina Gayathri
Register Number	192472074
Course	Programming in Java
Assignment	Assignment 5
Course Outcome	CO5 – Data Persistence Engineering
Java Version	26.0.1
Database	MySQL 8.0.46
JDBC Driver	MySQL Connector/J

2. Introduction

This assignment studies persistence techniques in Java using files, buffering, DAO design, serialization, JDBC, transactions, batch processing, metadata, and file-to-database migration. Part A focuses on persistence design and pseudocode, while Part B implements the designs and evaluates them experimentally.

3. Objectives

• Design a suitable file format for bus-pass records.

• Analyze the effect of buffering on file I/O performance.

• Design a generic DAO abstraction for database access.

• Study object graph serialization and transient data.

• Apply JDBC transactions and safe parameterized SQL.

• Implement and test a file-based bus-pass store.

• Investigate adversarial input and malformed records.

• Compare Statement, PreparedStatement, and batch processing.

• Measure transaction performance and verify rollback behavior.

• Migrate bus-pass records from a file store into MySQL and inspect database metadata.

4. Part A – Persistence Design and Pseudocode

4.1 A1 – File Format Design

The bus-pass record contains pass ID, registration number, name, route number, boarding stop, validity date, and amount paid. A delimited text representation was considered for readable line-oriented storage, while a binary DataOutputStream representation was considered for compact typed storage. The design also considers delimiter handling, damaged records, missing files, incomplete writes, and fixed-length RandomAccessFile records.

The main operations are adding a record, finding a record by pass ID, and listing records for a route. A sequential scan is O(n), while a fixed-position RandomAccessFile lookup can be O(1) when the record position is known.

4.2 A2 – Buffering Analysis

A 50 MB attendance log read one byte at a time causes a very large number of read calls. BufferedInputStream reduces underlying I/O operations by reading larger blocks. Files.readAllLines() is simpler for whole-file processing but keeps all lines in memory.

4.3 A3 – DAO Design

A generic Dao abstraction separates persistence operations from application logic. Operations such as insert, findById, findAll, update, and delete can be implemented by StudentDB, CourseDB, and MarksDB classes. PreparedStatement is used for parameterized SQL.

4.4 A4 – Serialization

The Hostel → Room → Resident object graph demonstrates Java object serialization. A live database Connection should be transient because it should not be serialized. serialVersionUID supports class-version control. CSV or JSON is an alternative when language-independent storage is required.

4.5 A5 – Transactions and Safety

A wallet transfer of ■500 requires both debit and credit operations to succeed together. JDBC transaction control uses setAutoCommit(false), commit() on success, and rollback() on failure. The SQL-injection analysis contrasts unsafe string concatenation with PreparedStatement parameter binding.

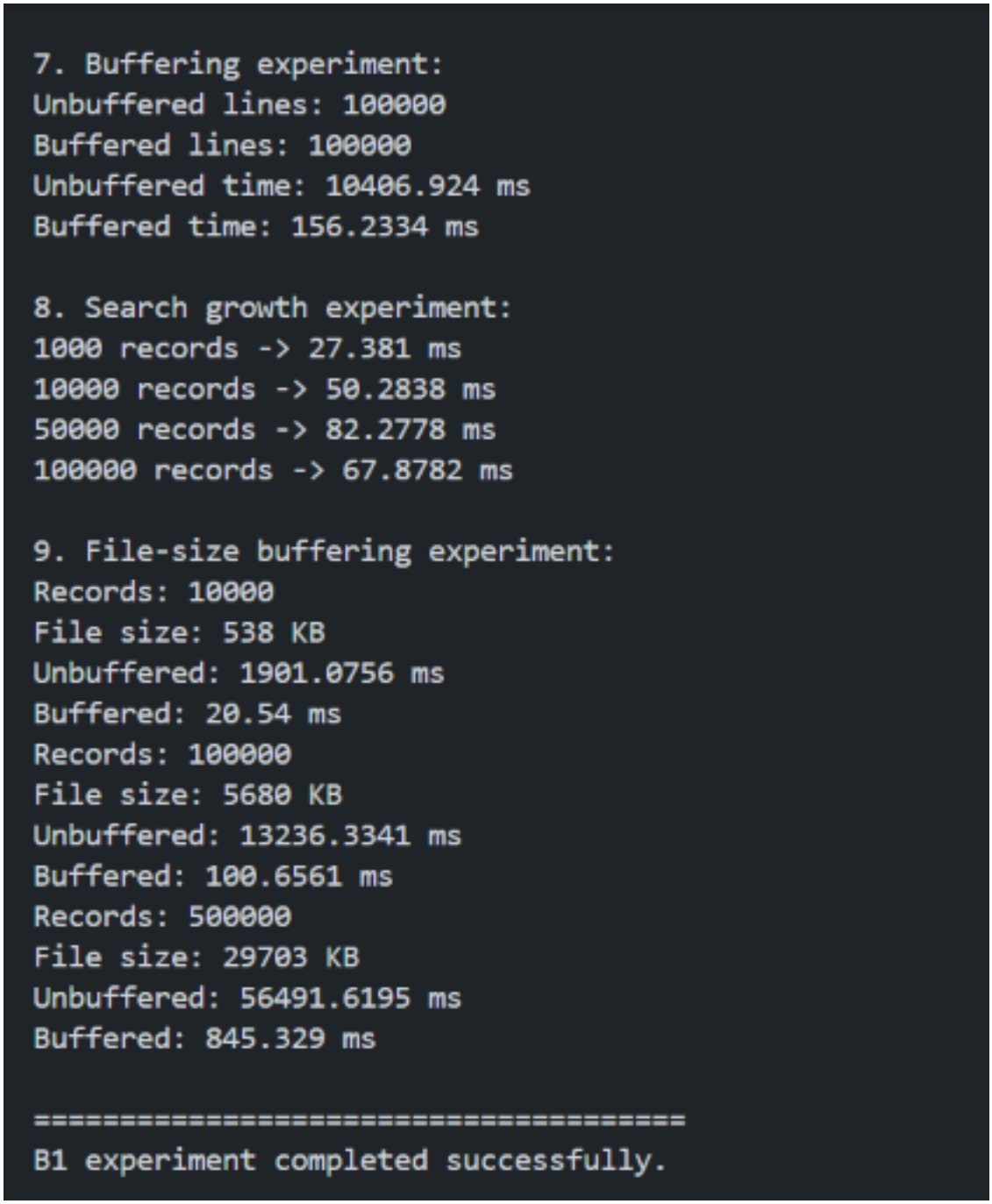
5. Part B – Implementation and Experimental Investigation

5.1 B1 – File Store

The B1 implementation provides bus-pass file operations including adding a pass, finding a pass by ID, listing passes by route, handling nonexistent IDs, and detecting damaged records. It also measures buffered versus unbuffered reading, search growth, and file-size effects.

Experiment	**Observed result**
Buffered lines	100000
Unbuffered time – captured run	10406.924 ms
Buffered time – captured run	156.2334 ms
Search growth	1000, 10000, 50000, 100000 records
File-size experiment	10000, 100000, 500000 records

B1 Output Screenshots



B1 Output – Buffering, Search Growth and File-Size Experiment

```
1. Test file created.
Pass added successfully.

2. Finding P100:
Pass found:
P100,192472074,Gayathri,101,Avadi,2026-12-31,500.0

3. Finding nonexistent ID:
Pass ID not found.

4. Passes for route 101:
P100,192472074,Gayathri,101,Avadi,2026-12-31,500.0

5. Passes for route 999:
No passes found for route 999

6. Buffering experiment:
Unbuffered lines: 100000
Buffered lines: 100000
Unbuffered time: 20778.9043 ms
Buffered time: 136.306 ms

7. Search growth experiment:
1000 records -> 12.3373 ms
10000 records -> 20.4943 ms
50000 records -> 79.3105 ms
100000 records -> 109.4005 ms
```

B1 Output – Functional Tests and Initial Experiments

```
1. Test file created.
Pass added successfully.

2. Finding P100:
Pass found:
P100,192472074,Gayathri,101,Avadi,2026-12-31,500.0

3. Finding nonexistent ID:
Pass ID not found.

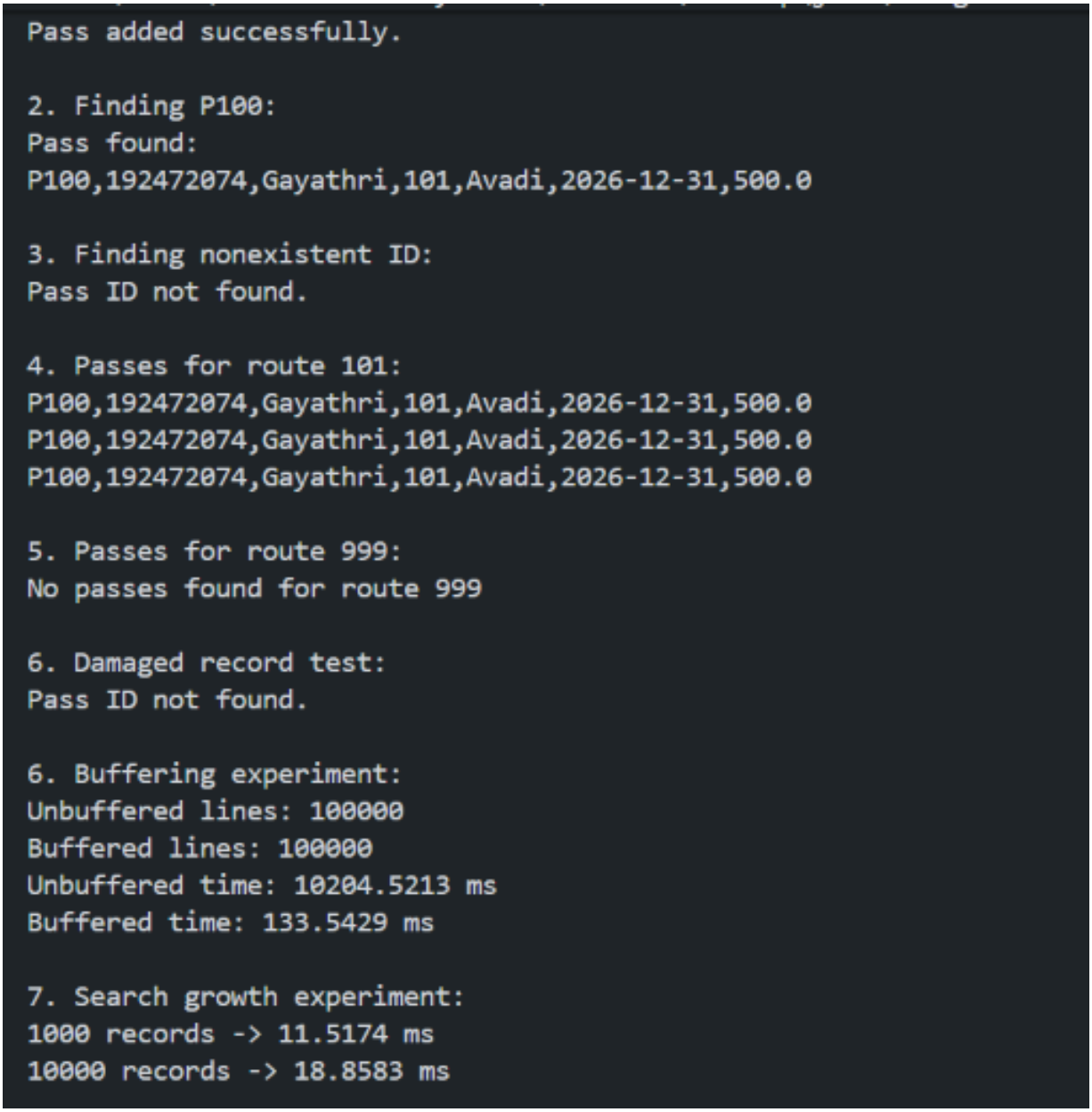
4. Passes for route 101:
P100,192472074,Gayathri,101,Avadi,2026-12-31,500.0
P100,192472074,Gayathri,101,Avadi,2026-12-31,500.0

5. Passes for route 999:
No passes found for route 999

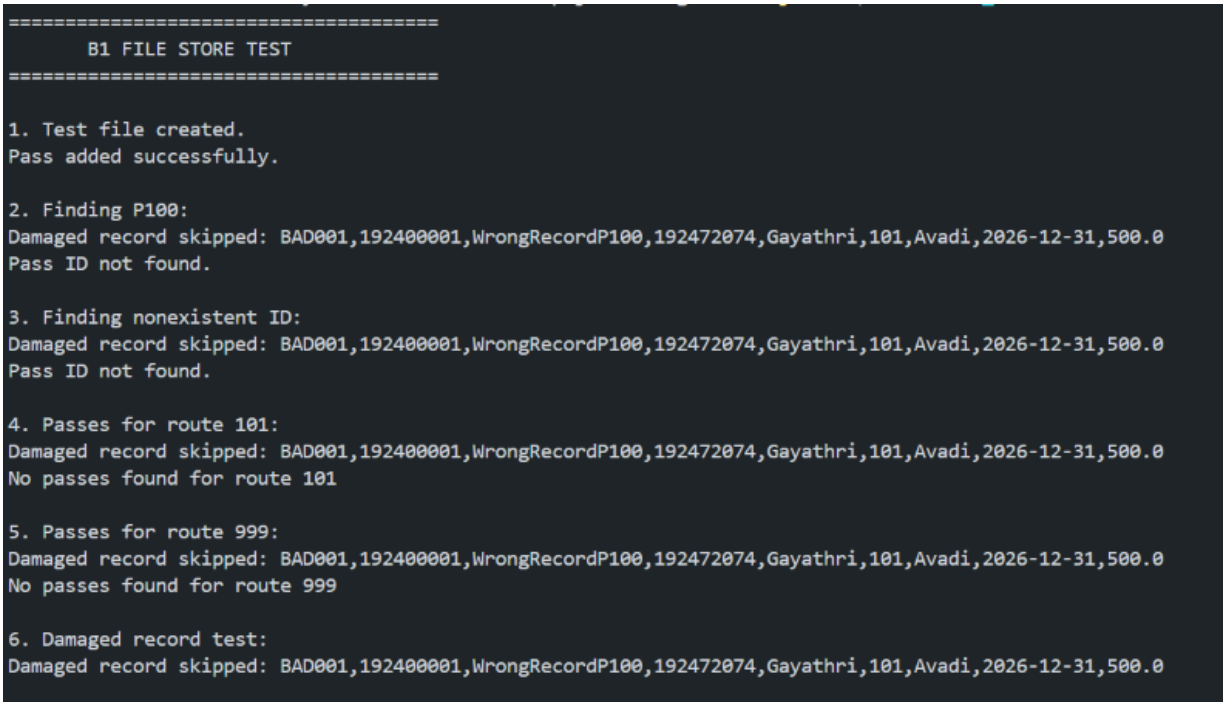
6. Buffering experiment:
Unbuffered lines: 100000
Buffered lines: 100000
Unbuffered time: 10138.7745 ms
Buffered time: 138.7134 ms

7. Search growth experiment:
1000 records -> 8.1363 ms
10000 records -> 21.7367 ms
50000 records -> 56.0813 ms
100000 records -> 45.531 ms
```

B1 Output – Functional Tests and Search Growth



B1 Output – Damaged Record Test



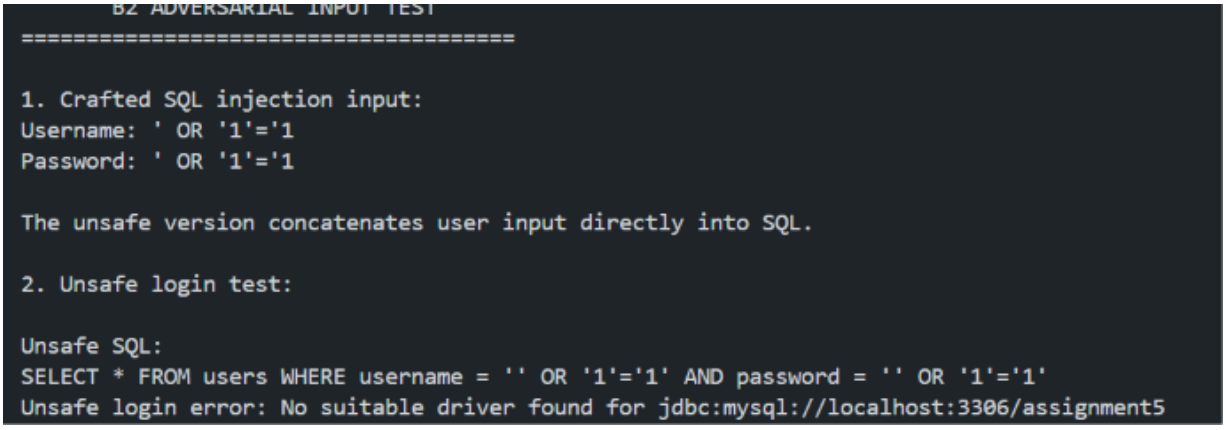
B1 Output – Damaged Record Detection

5.2 B2 – Adversarial Input

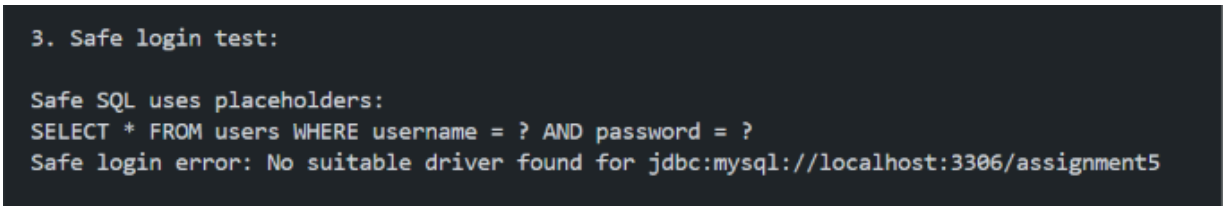
The B2 implementation tests a crafted SQL-injection input against unsafe SQL construction and contrasts it with PreparedStatement placeholders. It also tests malformed bus-pass records containing wrong field counts, a non-numeric amount, an empty line, and an invalid route.

The captured output shows the unsafe SQL being constructed directly from input, while the safe version uses placeholders. The damaged-file test identifies the malformed lines and their input problems.

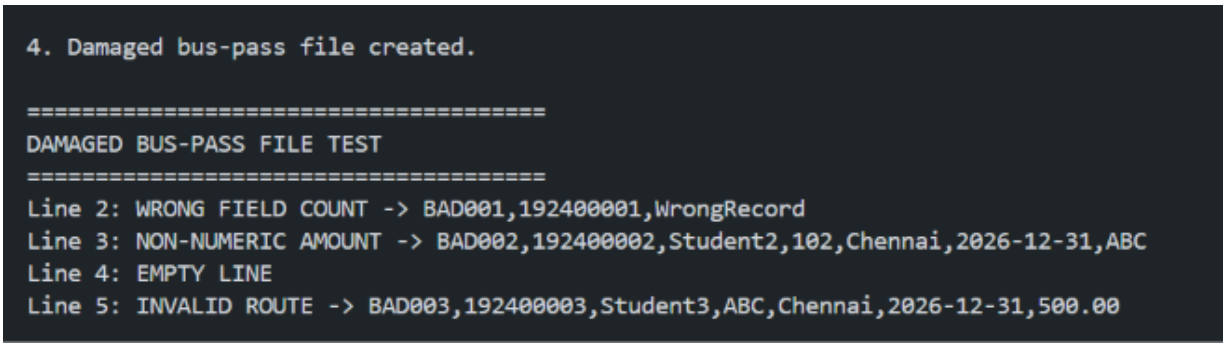
B2 Output Screenshots



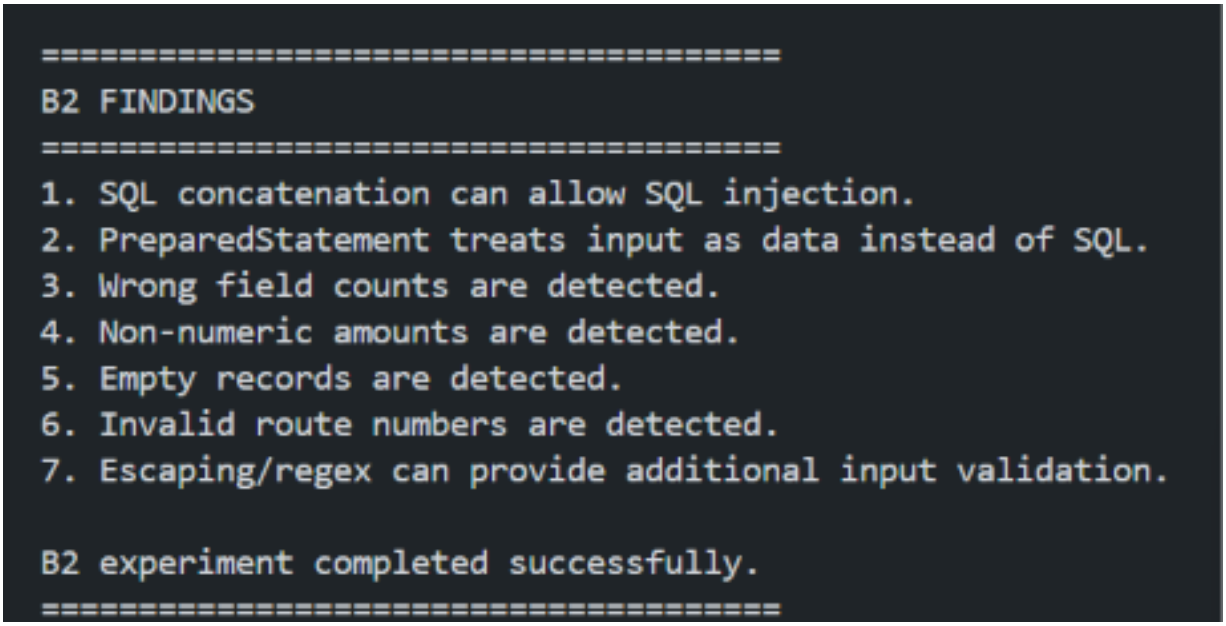
B2 Output – Crafted SQL Injection and Unsafe Login



B2 Output – PreparedStatement Login



B2 Output – Damaged Bus-Pass Input



B2 Output – B2 Findings

5.3 B3 – JDBC Experiments

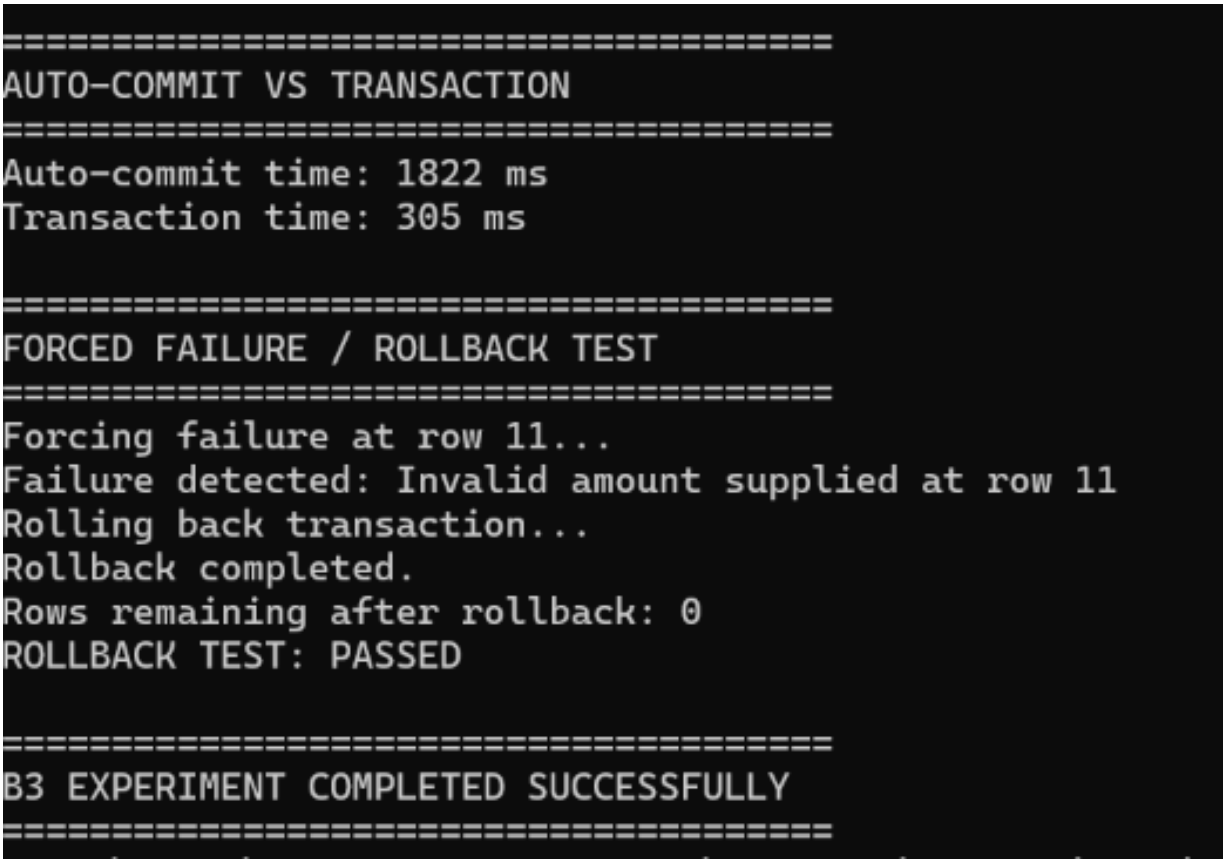
The B3 implementation connects to the assignment5 MySQL database, performs a 10-row correctness test, reads ResultSetMetaData, benchmarks Statement, PreparedStatement, and batch processing, compares auto-commit with a transaction, and forces a failure to verify rollback.

Rows	Statement (ms)	PreparedStatement (ms)	Batch (ms)
1,000	2071	2134	2903
10,000	16653	16658	15645
100,000	178936	173481	186995

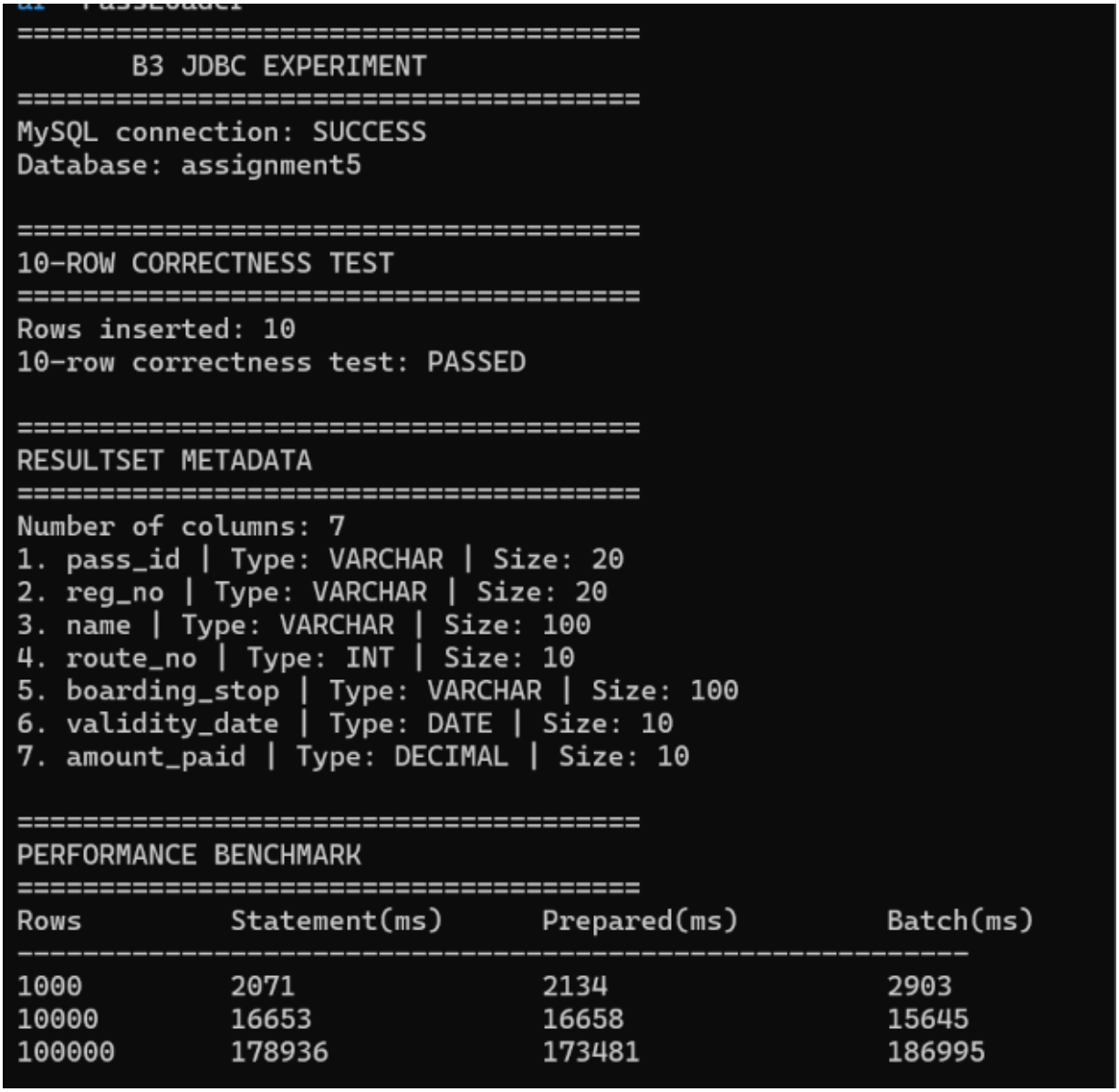
Mode	Measured time
Auto-commit	1822 ms
Transaction	305 ms

ResultSetMetaData reports seven columns: pass_id, reg_no, name, route_no, boarding_stop, validity_date, and amount_paid. The forced-failure test reports zero rows remaining after rollback.

B3 Output Screenshots



B3 Output – JDBC Connection, Correctness, Metadata and Performance



B3 Output – Transaction and Rollback Test

5.4 B4 – File-to-Database Migration

The B4 implementation migrates the bus-pass text file into MySQL using buffered file reading, PreparedStatement, batch processing, and transaction control. It also performs a failure/rollback test and retrieves DatabaseMetaData.

Result	**Observed output**
Connection	MySQL connection: SUCCESS
Database	assignment5
Migration	Migration committed successfully
Records migrated	10
Records skipped	0
Rollback test	PASSED
Records before failure test	10
Records after rollback	10
Database Product	MySQL
Database Version	8.0.46
JDBC Driver	MySQL Connector/J

B4 Output Screenshot



B4 Output – Migration, Rollback and Database Metadata

6. Experimental Discussion

The B1 output demonstrates the practical benefit of buffering for repeated file input operations. The captured runs show substantially lower buffered times than unbuffered times, although exact measurements vary between runs.

The B3 measurements show that the relative performance of Statement, PreparedStatement, and batch processing changes with dataset size. In the captured 10,000-row run, batch processing had the lowest measured time. In the 100,000-row run, PreparedStatement had the lowest measured time.

The transaction measurement was lower than the auto-commit measurement in the captured run. The rollback test demonstrates atomicity because no partial inserted rows remained after the forced failure.

7. Engineering Decision

For a bus-pass system spanning multiple campuses and supporting a larger number of records and concurrent operations, a relational database is the preferred persistence mechanism. MySQL provides structured storage, concurrent access, transactions, metadata, and query capabilities that are difficult to manage reliably with plain text files.

PreparedStatement combined with transaction-controlled batch processing is the preferred database implementation because it provides parameterized SQL, transactional consistency, and a suitable mechanism for repeated insertion. Multi-row INSERT or MySQL LOAD DATA can be considered for very large imports when their validation and operational requirements are appropriate.

A campus-local serialized-file approach remains an alternative for isolated offline operation, but is less suitable when centralized concurrent access and coordinated updates are required.

8. Conclusion

Assignment 5 demonstrates the persistence workflow from file-format design to database migration. The work covers file I/O, buffering, DAO abstraction, serialization, secure JDBC access, performance benchmarking, transaction control, rollback, database metadata, and migration. The experimental results support transaction-controlled PreparedStatement and batch processing for the final database-oriented design.

9. GitHub Repository

Repository: https://github.com/GAYATHRI190506/CSA09-192472074-Assignment5